

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 10

Regression in Banking

From First Principles to Modern AI — Pricing a Loan with Data.

Basic linear regression → ensembles → deep learning → 2026 tabular foundation models



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

From First Principles to the 2026 Frontier



1 • Foundations

Real banking data, cleaning, and the leakage trap



2 • Linear Models

Linear, Ridge, Lasso, ElasticNet



3 • Tree Ensembles

Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost



4 • Deep Learning

A PyTorch neural network for tabular data



5 • Modern AI (2026)

Tabular foundation models + GenAI explanations

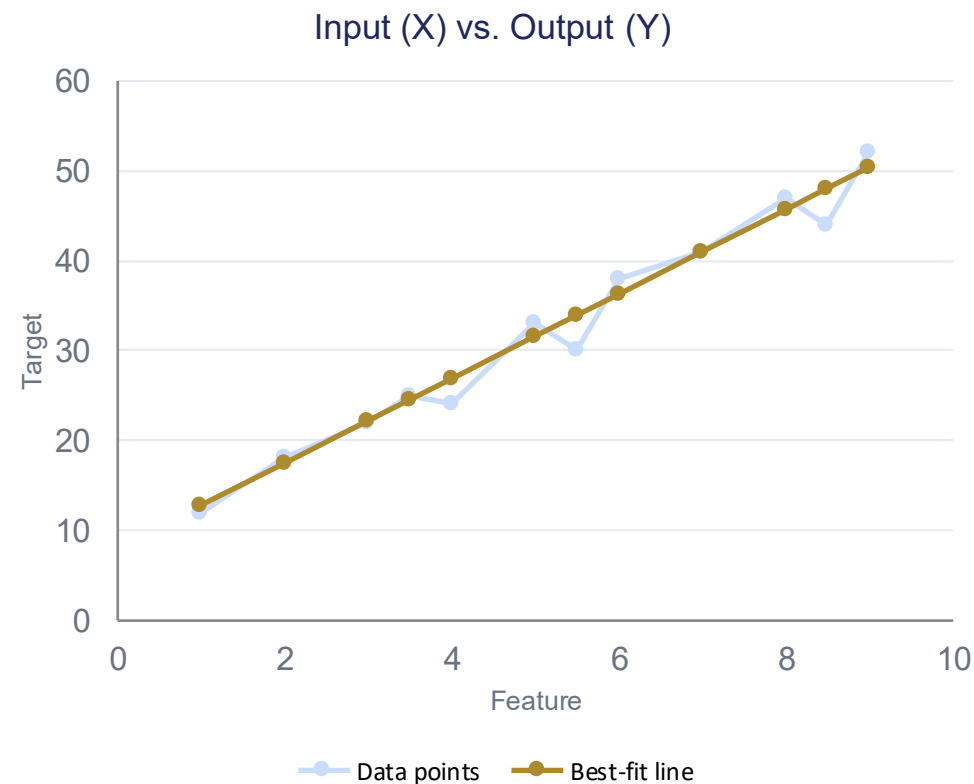


6 • Explainability

SHAP, permutation importance, partial dependence

What Is Regression, Really?

- Regression predicts a **continuous number** from a set of inputs — not a category.
- Classification asks “which bucket?” (spam / not spam). Regression asks “what value?” (37.2, \$410,000, 14.6%).
- Every regression model is really a function: given features X , output a number $\hat{y}$ as close as possible to the true value y .
- **Everyday examples:** house price from square footage, temperature tomorrow from today’s weather, exam score from hours studied.
- **Our example today: the interest rate a bank should charge a loan applicant.**



Why a Bank Needs Regression

Every loan application forces the same question:

“What interest rate compensates us for the risk of lending this person money?”



Price too high

Good borrowers walk away to a competitor platform.



Price too low

The platform loses money when risky loans default.



Price it right

Regression learns the pricing function from historical loans.

Meet the Dataset

2,199

training loans

300

held-out applicants

12

usable features

Feature	What it captures
Amount.Requested	How much the applicant wants to borrow
FICO.Range → FICO.Mid	Credit score band, midpoint (e.g. “720-724” → 722)
Debt.To.Income.Ratio	Existing debt load relative to income
Monthly.Income	Applicant's monthly income
Loan.Length	36 or 60-month term
Loan.Purpose, State, Home.Ownership	Categorical context features
Employment.Length	Job tenure, from “< 1 year” to “10+ years”
Interest.Rate	TARGET — what we are predicting

Cleaning: The Part Nobody Tells You About

Real bank data is never analysis-ready. Roughly 90% of a real regression project is here — before any model gets touched.

`"18.49%"`



`18.49`

Strip the % sign, cast to float

`"720-724"`



`722.0`

Use the midpoint of the FICO band

`"60 months"`



`60.0`

Strip the word "months"

`"10+ years" / "< 1 year" / "n/a"`



`10.0 / 0.5 / missing`

Custom mapping per category

`Literal "." in a numeric column`



`NaN → imputed`

Treat as missing, not a value

Data Leakage: When a Feature Cheats



- **Amount.Funded.By.Investors** looks like an innocent number — but investors decide how much to fund *after* seeing the quoted rate.
- It correlates **0.97** with Amount.Requested — nearly a duplicate feature, and temporally suspicious.
- **The rule to teach:** “Would I know this value BEFORE quoting the rate?” If not — it’s leakage.
- **A leaky model looks brilliant in the notebook and fails on the very first real applicant.**

Correlation Check

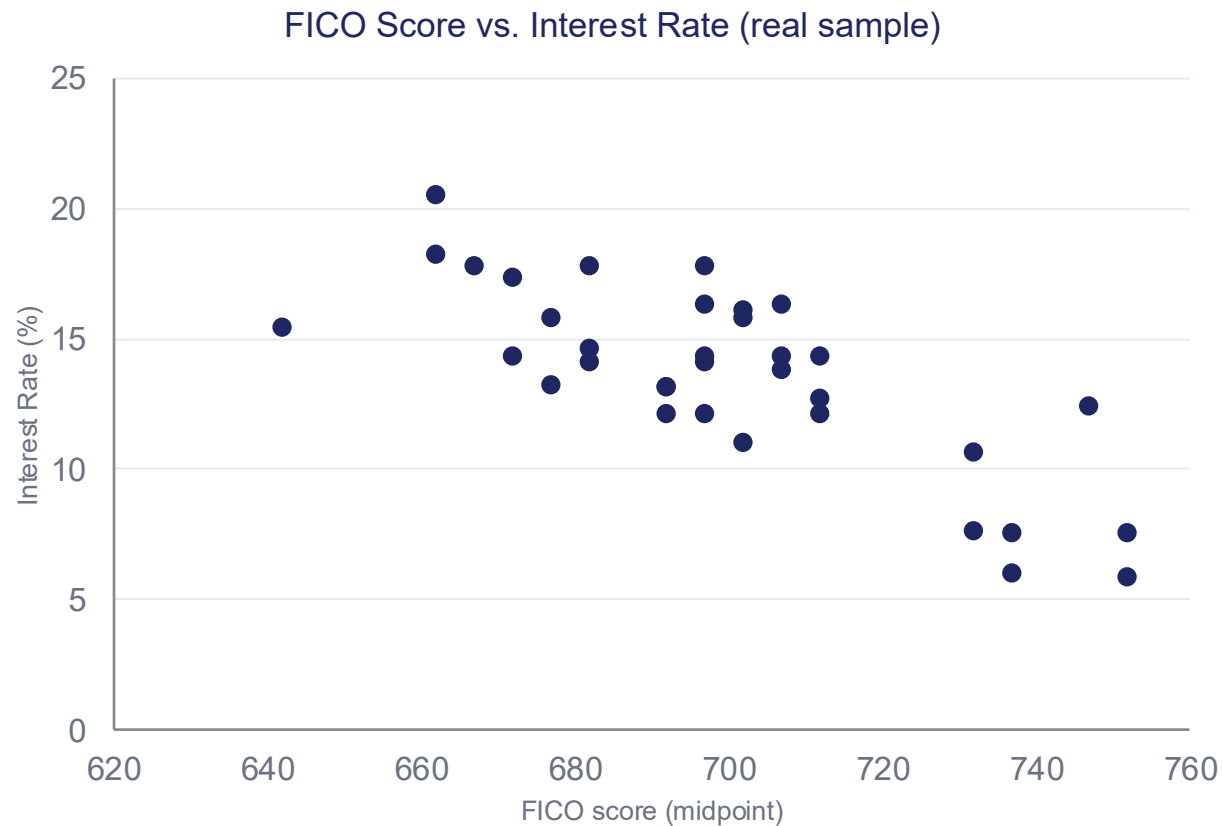
Amount.Funded.By.Investors ↔ Amount.Requested **0.97**

Amount.Funded.By.Investors ↔ Interest.Rate **0.34**

Amount.Requested ↔ Interest.Rate **0.33**

Decision: exclude it from our “fair” feature set.

What the Data Already Tells Us



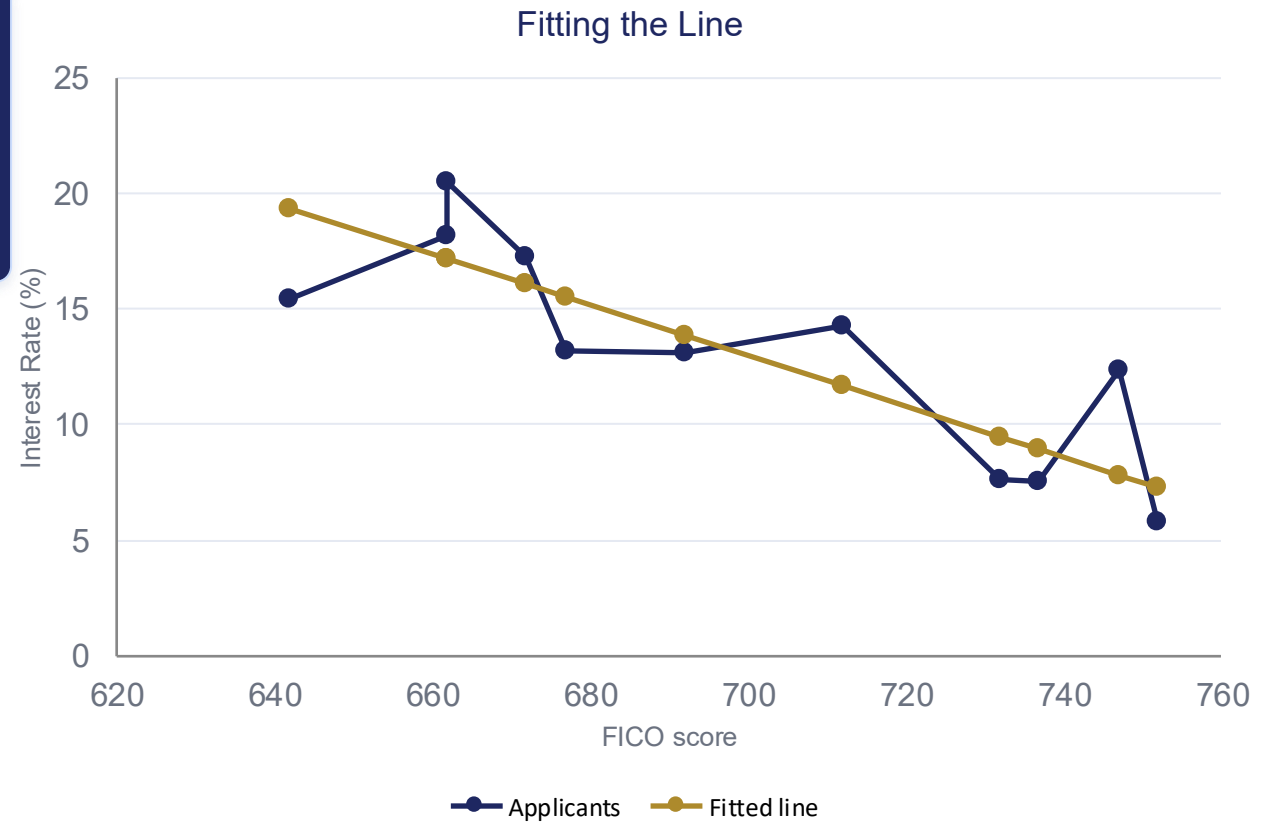
- **Higher FICO score** → lower interest rate — exactly the risk-based pricing we'd expect.
- Interest rates in this sample range roughly from 6% to 25%, and are roughly bell-shaped.
- 60-month loans tend to carry higher rates than 36-month loans (longer exposure, more risk).
- **Takeaway:** even before modeling, the data already hints at the pricing logic a model needs to learn.

Simple Linear Regression, from Scratch

$$\hat{y} = \beta_0 + \beta_1 \times \text{FICO}$$

predicted rate = intercept + slope × credit score

- β_0 , β_1 (the coefficients) are chosen to draw the **single best-fit line** through the data.
- “Best” means: **minimizing the sum of squared errors** (Ordinary Least Squares) — the classic 200-year-old idea behind almost all of regression.
- Every dot above the line is under-predicted; every dot below is over-predicted. Squaring the errors punishes big misses harder than small ones.



From One Feature to Many

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

- Real pricing depends on many things at once — not just FICO. We add loan amount, DTI, income, term, employment length, and one-hot encoded categories (purpose, state, home ownership).
- **Categorical features** (e.g. “RENT”, “MORTGAGE”) become 0/1 columns via one-hot encoding — the model still just multiplies and adds.
- The model automatically learns **which features matter most** and in which direction, from the training data.

Top Drivers — Actual Linear Regression Coefficients

State = IN	-4.41
Home Ownership = NONE	+3.15
FICO.Mid	-3.05
State = AK	+2.56
State = DC	-1.59
Loan.Length (months)	+1.31
Amount.Requested	+1.28

Linear Regression: Predicting an Outcome

Linear regression is a statistical method used to understand the relationship between:

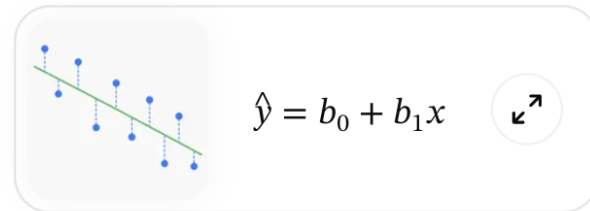
- **Target (Y):** what we want to predict
- **Features (X):** variables that may explain or predict Y

For our loan dataset:

Target: Interest Rate

Possible predictors: FICO Score, Debt-to-Income Ratio, Loan Amount, Income, Credit Lines, etc.

The basic equation is:



$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k$$

Example:

$$\text{Interest Rate} = \beta_0 + \beta_1(\text{FICO}) + \beta_2(\text{DTI}) + \beta_3(\text{Loan Amount})$$

The model tries to find the **best coefficients (β)** that explain the observed data.

$$\hat{y} = b_0 + b_1x$$

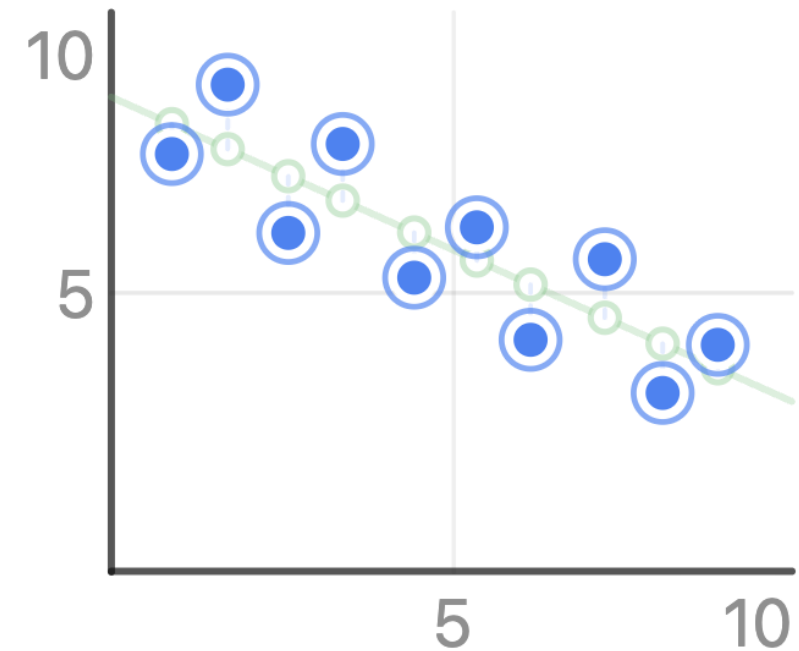
$$\hat{y} = 8.49 - 0.54x$$

$R^2 = 0.72$ · b_0 = intercept · b_1 = slope ·
least squares minimizes squared
vertical residual gaps.

Focus

Data ↕

Drag a point to refit the line.



Understanding Our Loan Dataset

Variable	Meaning	Type
Interest Rate	Loan interest rate	Target (Y)
FICO Range	Borrower's credit score	Predictor
Amount Requested	Requested loan amount	Predictor
Debt-to-Income Ratio	Debt relative to income	Predictor
Monthly Income	Borrower's monthly income	Predictor
Open Credit Lines	Number of open credit accounts	Predictor
Revolving Credit Balance	Outstanding revolving debt	Predictor
Loan Length	36 or 60 months	Predictor
State	Borrower's state	Categorical
Home Ownership	Rent / Own / Mortgage	Categorical

Key idea:

We use information available **before the loan decision** to predict something about the loan.

In banking, this is important because we want the model to work on **new applicants**, not just explain historical data.

Coefficient = How Much Y Changes When X Changes

Suppose our model gives:

$$\text{Interest Rate} = 25.0 - 0.02(\text{FICO}) + 0.15(\text{DTI}) + 0.0001(\text{Loan Amount})$$

The coefficients tell us the **direction and size of the relationship**, holding other variables constant.

Example: FICO coefficient = -0.02

If FICO increases by **1 point**:

Predicted Interest Rate decreases by approximately **0.02 percentage points**, all else equal.

If FICO increases by 50 points:

Interest Rate decreases by approximately **1 percentage point**.

How to read coefficients

- **Positive coefficient** → higher X → higher predicted Y
- **Negative coefficient** → higher X → lower predicted Y
- **Large absolute coefficient** → stronger effect *depending on the variable's scale*

⚠ **Important:** Coefficient ≠ causation.

It tells us about the relationship estimated by the model.

From Data → Model → Decision

A bank could use regression to estimate:

Expected Interest Rate

For a new borrower:

Input

- FICO = 720
- DTI = 15%
- Loan Amount = \$20,000
- Income = \$6,000/month

↓

Regression Model

$$\widehat{\text{Interest Rate}} = \beta_0 + \beta_1 \text{FICO} + \beta_2 \text{DTI} + \dots$$

↓

Prediction

Predicted Interest Rate = X%

The model can help the bank:

- Price loans
- Estimate expected risk
- Compare borrowers consistently
- Support underwriting decisions
- Identify important risk drivers

But: the model should be used as **decision support**, not blindly followed.

Typical Regression Modeling Process

1. Prepare the data

Clean missing values and convert variables into usable formats.

Examples:

- "18.49%" → 18.49
- "720-724" → approximately 722
- "60 months" → 60

2. Split the data

Training data → build the model

Test data → evaluate the model on unseen observations

3. Train regression

Use historical loans to estimate:

$$\beta_0, \beta_1, \dots, \beta_k$$

4. Predict

Give the model a new borrower → generate predicted interest rate.

5. Evaluate

Check whether predictions are actually accurate.

Very important: Don't evaluate the model only on the data used to train it.

Model Evaluation

For regression, common metrics include:

MAE — Mean Absolute Error

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Simple interpretation:

How many percentage points are we wrong on average?

Example:

MAE = 1.2%

→ Our predictions are off by about **1.2 percentage points on average**.

RMSE — Root Mean Squared Error

RMSE penalizes large mistakes more heavily.

Useful when big prediction errors are particularly undesirable.

R² — R-squared

Measures how much variation in Y is explained by the model.

Example:

R² = 0.70

Roughly means the model explains **70% of the variation** in the target within this modeling context.

Ridge, Lasso & ElasticNet

Plain linear regression can fit noise in the training data. Regularization adds a penalty on large coefficients to keep the model honest.



Ridge (L2)

Shrinks all coefficients smoothly toward zero — keeps every feature, dampens noise.

$$R^2 = 0.751$$



Lasso (L1)

Can push weak coefficients all the way to zero — built-in feature selection.

$$R^2 = 0.763$$



ElasticNet

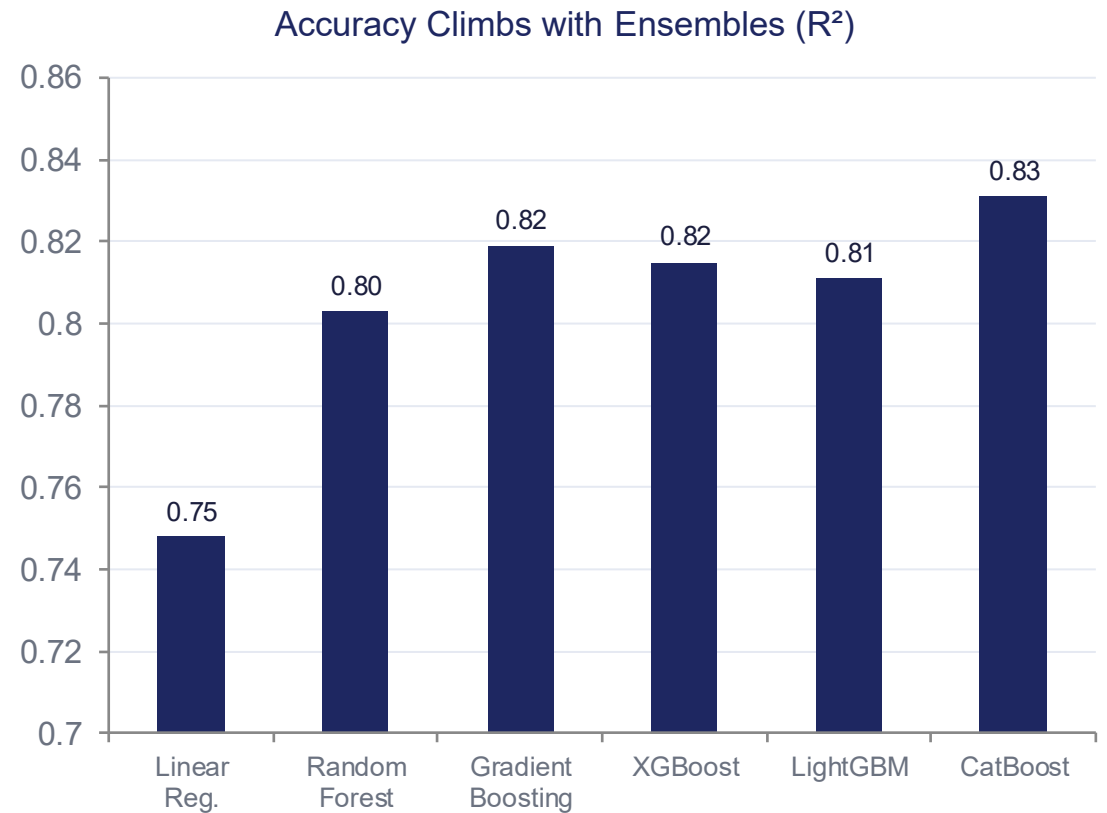
Blends both penalties — a practical middle ground.

$$R^2 = 0.762$$

Baseline for comparison: plain Linear Regression scored $R^2 = 0.748$ — regularization nudges it higher by fighting overfitting.

Leveling Up: Trees & Ensembles

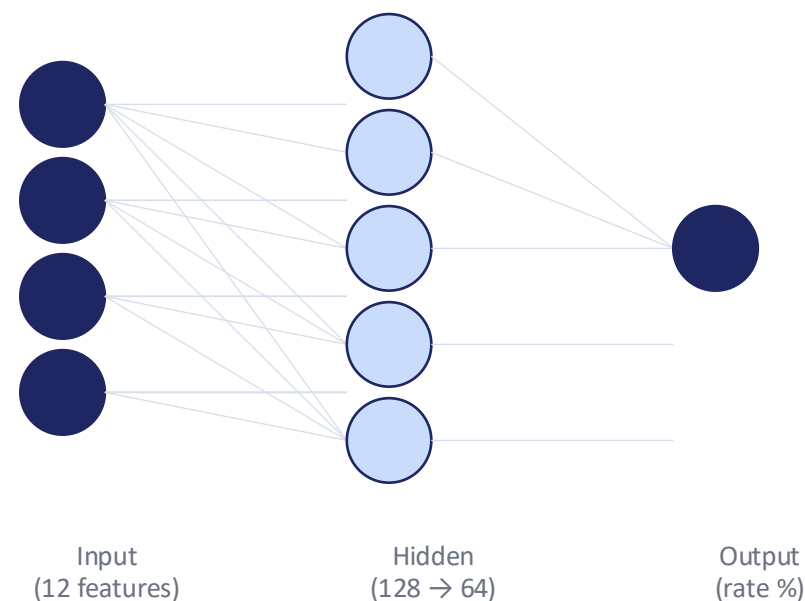
- **Decision Tree:** splits the data on the feature that reduces error the most, again and again.
- **Random Forest:** trains many trees on random subsets and averages them (bagging) — reduces variance.
- **Gradient Boosting family:** XGBoost, LightGBM, CatBoost build trees *sequentially*, each correcting the previous ones' mistakes.
- This family usually wins **on structured, tabular data** like ours — exactly the kind banks work with.



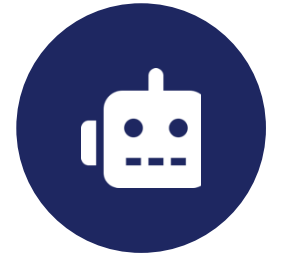
Going Deep: A Neural Network for Tables



- A small feedforward network (MLP) can learn non-linear interactions between features without us hand-engineering them.
- **Architecture:** input layer $\rightarrow$ 128 neurons (ReLU) $\rightarrow$ 64 neurons (ReLU) $\rightarrow$ 1 output neuron (the predicted rate).
- Trained with **PyTorch** using the Adam optimizer, minimizing Mean Squared Error — on a free Colab GPU.
- Dropout layers randomly silence neurons during training to prevent overfitting — a form of regularization, just like Ridge/Lasso for linear models.



The 2026 Frontier: Tabular Foundation Models



- **TabPFN** is pretrained once on millions of synthetic datasets and shared via Hugging Face.
- **No training loop on your data** — it makes predictions on a brand-new table with a single forward pass.
- This is **in-context learning** — the same mechanism behind ChatGPT-style prompting, applied to spreadsheets instead of text.
- It won't always beat a tuned CatBoost — but for small tabular problems with no time to tune five models, it is a genuinely relevant 2026 tool.

Explaining Predictions in Plain English



Key principle: the regression model still makes the actual pricing decision. GenAI never touches the number — it only translates the model's own explanation into human language. This separation is exactly how responsible AI is built in regulated industries like banking.

Six Error Metrics, Not Just One

No single metric tells the whole story — every model in our lab is graded on all six at once.

MAE

Average error in percentage points — easy to explain to a manager.

MSE

Like MAE, but squares errors — punishes big misses harder.

RMSE

Square root of MSE — back in the target's original units (%).

MAPE

Error as a % of the true rate — comparable across price levels.

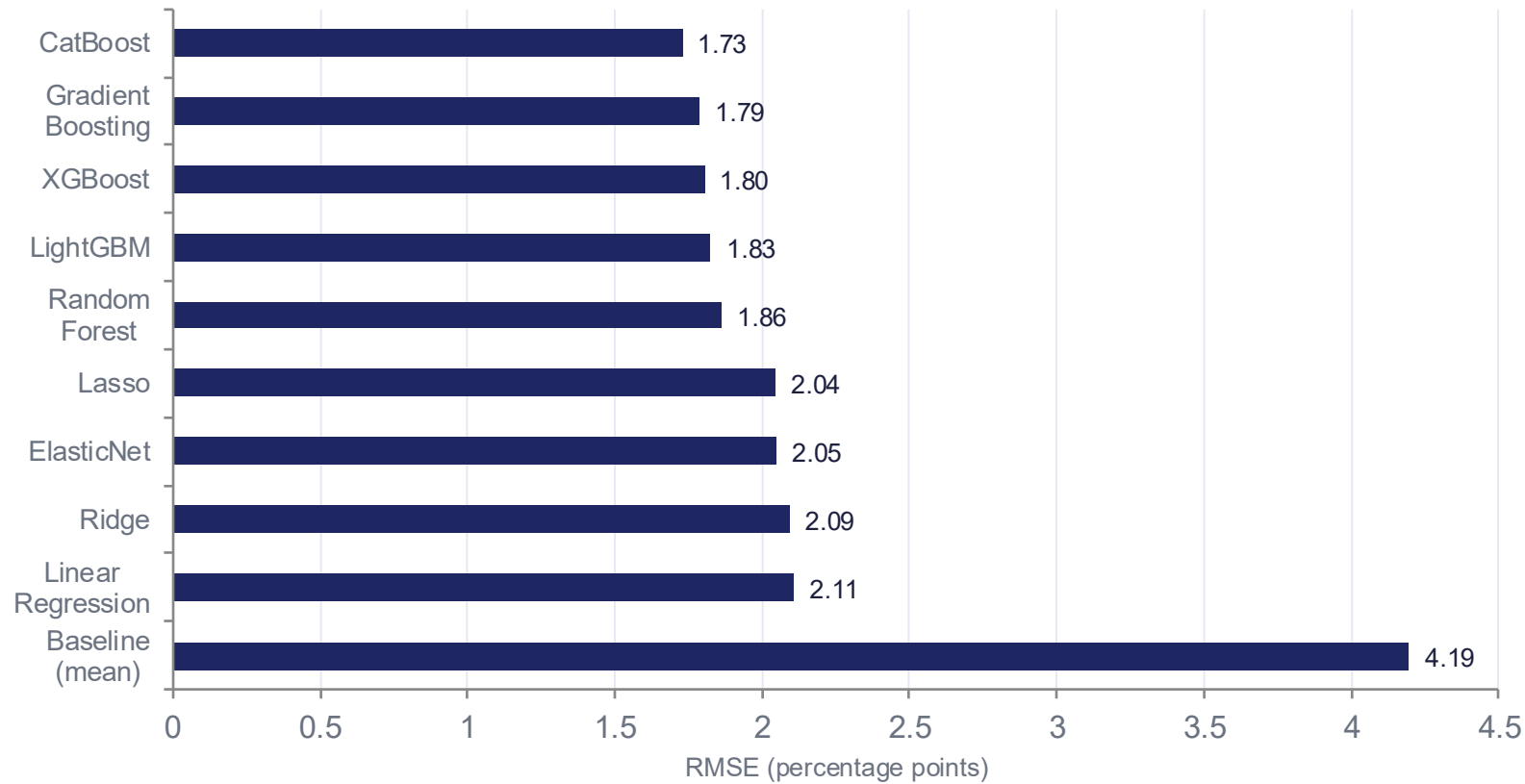
R^2

Fraction of variance explained. 1.0 = perfect, 0 = no better than guessing the mean.

Median AE

Typical error, robust to a few very bad predictions.

The Leaderboard: Every Model, One Scoreboard



Winner

CatBoost

R^2 **0.831**

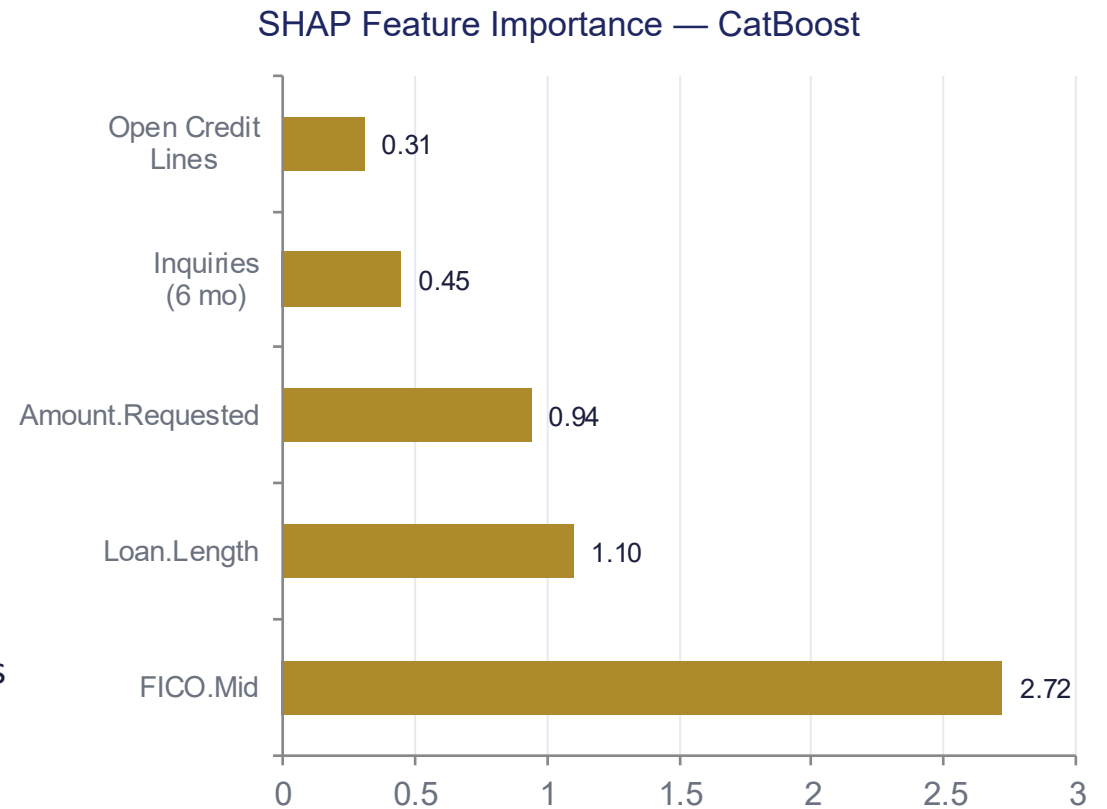
MAE **1.24 pp**

MAPE **10.2%**

vs. baseline **-58.8% RMSE**

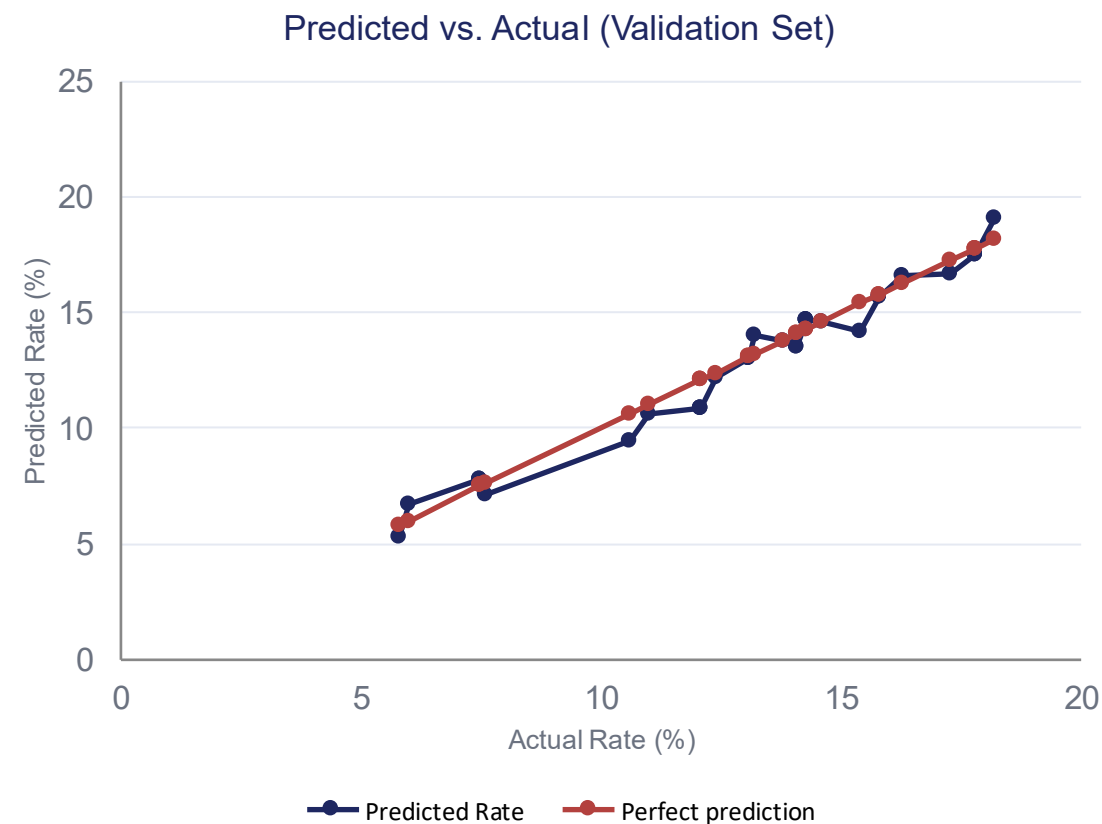
Why Did the Model Say That?

- **Permutation importance:** shuffle one column, see how much validation error gets worse — model-agnostic ground truth.
- **SHAP values:** attribute each prediction's deviation from average to individual features, with sign and size.
- Top driver, every time: **FICO score** — exactly matching how these platforms actually price risk.
- **Regulatory angle:** the U.S. Equal Credit Opportunity Act requires lenders to be able to explain an adverse pricing decision — explainability isn't optional in banking.



Inference in Practice: Quoting a Real Applicant

- **Step 1 — Grade yourself:** on the validation set we know the true rate, so predicted-vs-actual reveals real accuracy.
- A tight cloud along the diagonal = well-calibrated pricing across cheap and expensive loans alike.
- **Step 2 — Deploy:** on the 300 held-out test applicants, nobody — including us — knows the true rate. We just quote it.
- On average, our quotes miss the true rate by only **1.24 percentage points** (MAE).



Five Rules Worth Remembering

1

Always compare to baseline first.

A model that barely beats “guess the average” isn’t earning its complexity.

2

RMSE vs. MAE gap = outlier check.

A much bigger RMSE than MAE means a few applicants are badly mispriced.

3

Linear-vs-tree gap reveals structure.

A small gap means the relationship is mostly linear — extra complexity is optional.

4

Explainability is required, not optional.

In lending, regulations like ECOA demand you can explain an adverse decision.

5

Re-check leakage on every new feature.

The single most common way real-world models fail silently in production.

Where to Go From Here



Push TabPFN further

Run it on the full training set with a real GPU — does it catch CatBoost?



Explain a single applicant

Feed one person's SHAP values into the GenAI layer instead of the model's overall top features.



Study fair lending in AI

Read on ECOA, disparate impact, and explainable AI requirements in regulated finance.



Try causal thinking

Correlation prices risk; causal inference asks what changing a feature would actually do to the rate.



Extend to default risk

Reframe as classification: will this loan default? Compare regression and classification framings.



Build your own dataset

Apply this whole pipeline to a new banking problem — fraud scores, credit limits, churn.



Thank You!

Questions? Let's talk regression, banking, and where AI is headed in 2026.

Dataset & Notebook:

github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1